

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of The University of Newcastle pursuant to Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

SENG1050

Web Technologies

Week 7 - Content Management

Assessment 1 (Mid-Semester Quiz)

- ❑ Takes place on 12 Sep (Friday this week). The online quiz will open from 8 am to 11:59 pm.
- ❑ Assessment time: 90 minutes (answering) + 10 minutes (checking). All quiz sessions will end at 11:59 pm, so if you start the quiz after 10:19 pm, you won't get the full time.
- ❑ Access via Canvas – Modules -> Assessments -> Mid-Semester Quiz. The quiz is open-book, however, all answers must be in your own words. Use of generative AI (such as ChatGPT) and plagiarism is strictly forbidden (all submissions will be checked).

Quiz format & coverage

- ❑ **Coverage:** Weeks 1-5.
- ❑ 15 questions (30 including sub-questions): mix of matching, multiple choice and short answer/essay (may include simple coding).
- ❑ Study slides marked with stars.



Recall from last week: The Client-Server Model

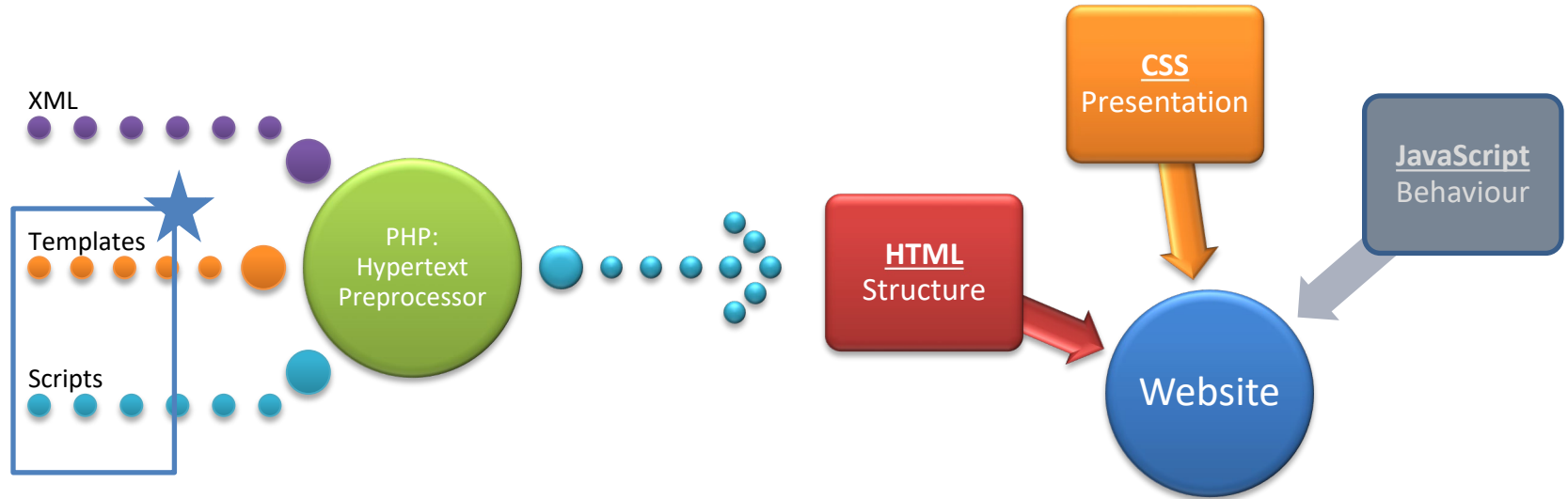
- ❑ Last week we completed our initial goal! We can now create a website on the internet, accessible using the world wide web. Of course, there's more to it than that, we also want a website that can scale, that is responsive, dynamic, fast, efficient, highly ranked by search engines, and secure. Our new goal is to create just such an efficient, optimised website on the internet, accessible using the world wide web.
- ❑ We'll need to split this goal into several parts: Scalability, Dynamic Behaviour, Responsiveness, Speed, Search Engine Optimisation, and Security are all big topics that we will need to cover separately. Additionally, we still have a couple of loose ends from the first half of the course, such as pulling our data from XML files into a HTML document.
- ❑ Today, we will look at the issue of scalability, and also find out how we can pull data from XML files into HTML documents.

Lecture Outline

- ❑ Content Management Systems (CMS)
- ❑ Simple PHP Template
- ❑ Xpath
- ❑ Writing XML-Connected Templates with PHP /
Converting XML to HTML

Content Management Systems (CMS)

The Server-Based Model





Motivation: Website Scalability

- ❑ As a website grows, it becomes increasingly difficult to keep track of content. Managing hundreds, thousands of webpages, stylesheets, images, and other types of content that may have multiple file types and potentially multiple revisions requires a system to keep categorise content to allow for easy searching, indexing, and version control. It is not feasible for a human to organise this manually, for larger websites there is simply too much content to keep track of.
- ❑ For larger websites, it is likely that multiple people will need to be able to create content, not all of whom may have experience with HTML. Additionally, these people may require differing levels of access to the website, based on their role. The security principle of least privilege tells us that each user should only have the minimum level of access required to carry out their job.



Content Management Systems (CMS)

- ❑ A Content Management System (CMS) is an application that manages the lifecycle of web content, including content creation, modification and updates, viewing, archival, and deletion.
- ❑ A CMS has multiple functions that are designed to be used by non-technical users for the purpose of managing websites. Most are extensible, meaning that additional functionality can be easily added through the use of optional modules or plugins.
- ❑ Some commonly used content management systems are Wordpress (currently the most popular – used by over ~40% of active websites), Joomla, Squarespace, Drupal, Wix, and Shopify.

CMS Functionality

- ❑ Most content management systems provide some common functionality such as:
 - ❑ Document Management: Handles the storage and indexing of web content to allow for searching, categorisation, and version management.
 - ❑ Content Creation: Allows non-technical users to create content, usually with the aid of a more intuitive interface such as a WYSIWYG (What You See Is What You Get) editor. Many systems come with additional tools to aid in optimising webpages for search engine indexing.
 - ❑ Access Control: Supports multiple users, allowing differing levels of access to parts of the website, web content, and management tools.
 - ❑ Templating: Manages the appearance of a webpage using a common collection of styles, page templates, and components to create a common appearance for all pages on the website. Users can swap templates to quickly change the look of the entire website.
 - ❑ Caching: Improves the performance of the website by caching static versions of frequently requested content, only regenerating them when the content is changed.



CMS Components

- ❑ Content Management Systems are comprised of two main components, a Content Management Application, and a Content Delivery Application:
- ❑ Content Management Application (CMA):
 - ❑ Provides the administrative interface that allows users to manage the website and content.
 - ❑ Acts as an abstraction layer that hides the technical components of the website from non-technical users, facilitating intuitive management without requiring technical knowledge.
 - ❑ Interacts with the data storage elements of the website to organise and store content.
 - ❑ Usually located in a special directory in most content management systems, such as /admin (malicious actors know the default names of these directories so it is generally wise to change them as soon as possible).



CMS Components

- ❑ Content Delivery Application (CDA):
 - ❑ Responsible for compiling the content and displaying it to visitors.
 - ❑ Provides the front-end of the website, accessible to the public (must not perform trusted operations: read-only access to data).
 - ❑ Implemented with server-side programming in languages such as PHP, Perl, Ruby, C, Python, etc.
 - ❑ Caches compiled web content to speed up future access (content is recompiled when updated).
 - ❑ Content is compiled based on pre-defined **templates** that determine the layout and styling of the resulting webpages.



CMS Templates

- ❑ Remember – we use external CSS files to provide common visual styling across multiple webpages. In a similar manner, webpages tend to also have common structural elements, for example, they may share the same navigation bar, header, footer, sidebar, and even content structure. We don't want to have to write the same HTML code for every single webpage, this is inefficient, and makes the website hard to change if one of these components needs to be updated.
- ❑ To solve this problem, we can use templates. Most content management systems include a web templating system, which allows pages to be generated based on common HTML elements, while customising only those elements that differ between pages.



CMS Web Templating Systems

- ❑ Web templating systems use three core elements:
 - ❑ A template source: One or more web templates that specify the common HTML elements for a particular page or category of pages on a website. It will generally include placeholders where content can be inserted. These may be written in HTML, PHP, XSLT, or any number of other templating languages.
 - ❑ A data source: XML files, JSON files, a database, or any other method of storing data. This provides the content that differs between web pages.
 - ❑ The templating engine: The processing component that selects the correct template for a particular page request, retrieves and inserts the data from the data source, and outputs the result (usually a HTML document ready to be delivered to the client). This is usually PHP, Java, Python, Ruby, Perl, or similar languages.

GetSimple CMS Page Management

The screenshot shows the 'Page Management' section of the GetSimple CMS admin interface. The browser address bar indicates the URL is 'seng1050.com/admin/pages.php'. The interface has a dark header with the title 'SENG1050 Testbed' and a navigation bar with tabs for 'Pages', 'Files', 'Theme', 'Backups', and 'Plugins'. The 'Pages' tab is active.

Below the navigation bar, the 'Page Management' section is displayed. It includes a 'TOGGLE STATUS' button and a 'FILTER' button. A table lists the pages, with one entry visible: 'Welcome to GetSimple CE!' with the URL 'http://seng1050.com/' and the date 'Aug 17, 2025'. A search icon is next to the date. Below the table, it says '1 total pages'.

On the right side of the 'Page Management' section, there is a vertical list of actions: 'View All Pages', 'Create New Page', 'Menu Manager', 'Edit components', and 'Snippets'.

At the bottom of the page, the footer text reads '© 2009-2025 GetSimple CMS CE – Version 3.3.22' and the 'GS CE' logo is in the bottom right corner.

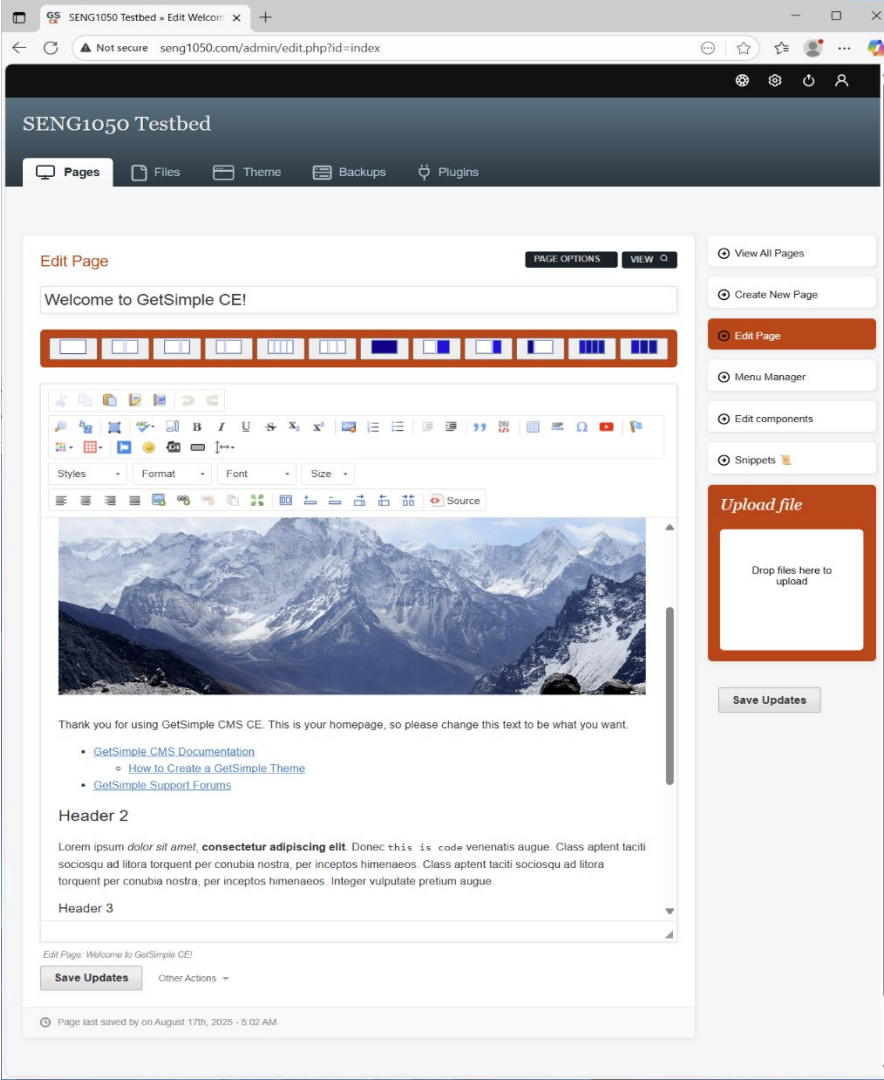
PAGE TITLE	URL	DATE
Welcome to GetSimple CE!	http://seng1050.com/	Aug 17, 2025

1 total pages

- View All Pages
- Create New Page
- Menu Manager
- Edit components
- Snippets

© 2009-2025 GetSimple CMS CE – Version 3.3.22

GetSimple CMS Page Editor (WYSIWYG Mode)



GetSimple CMS Page Editor (HTML Mode)

The screenshot shows the GetSimple CMS Page Editor in HTML Mode. The browser address bar indicates the URL is `seng1050.com/admin/edit.php?id=index`. The page title is "SENG1050 Testbed". The editor interface includes a top navigation bar with "Pages", "Files", "Theme", "Backups", and "Plugins". The main editing area is titled "Edit Page" and contains a "PAGE OPTIONS" button and a "VIEW" button. The content area displays the HTML code for the page, which includes a welcome message, a list of links to documentation, theme creation, and support forums, and a Lorem Ipsum paragraph. The editor also features a "Styles" and "Format" dropdown, a "Font" and "Size" dropdown, and a "Source" button. On the right side, there is a sidebar with options: "View All Pages", "Create New Page", "Edit Page" (highlighted), "Menu Manager", "Edit components", and "Snippets". Below these is an "Upload file" section with a "Drop files here to upload" instruction and a "Save Updates" button. At the bottom, there is a "Save Updates" button and a "Page last saved by on August 17th, 2025 - 5:02 AM" message.

SENG1050 Testbed

Pages Files Theme Backups Plugins

Edit Page

WELCOME TO GETSIMPLE CE!

Thank you for using GetSimple CMS CE. This is your homepage, so please change this text to be what you want.

- [GetSimple CMS Documentation](https://github.com/GetSimpleCMS-CE/GetSimpleCMS-CE/wiki)
- [How to Create a GetSimple Theme](https://github.com/GetSimpleCMS-CE/GetSimpleCMS-CE/wiki/Theme-Creation)
- [GetSimple Support Forums](https://github.com/GetSimpleCMS-CE/GetSimpleCMS-CE/discussions)

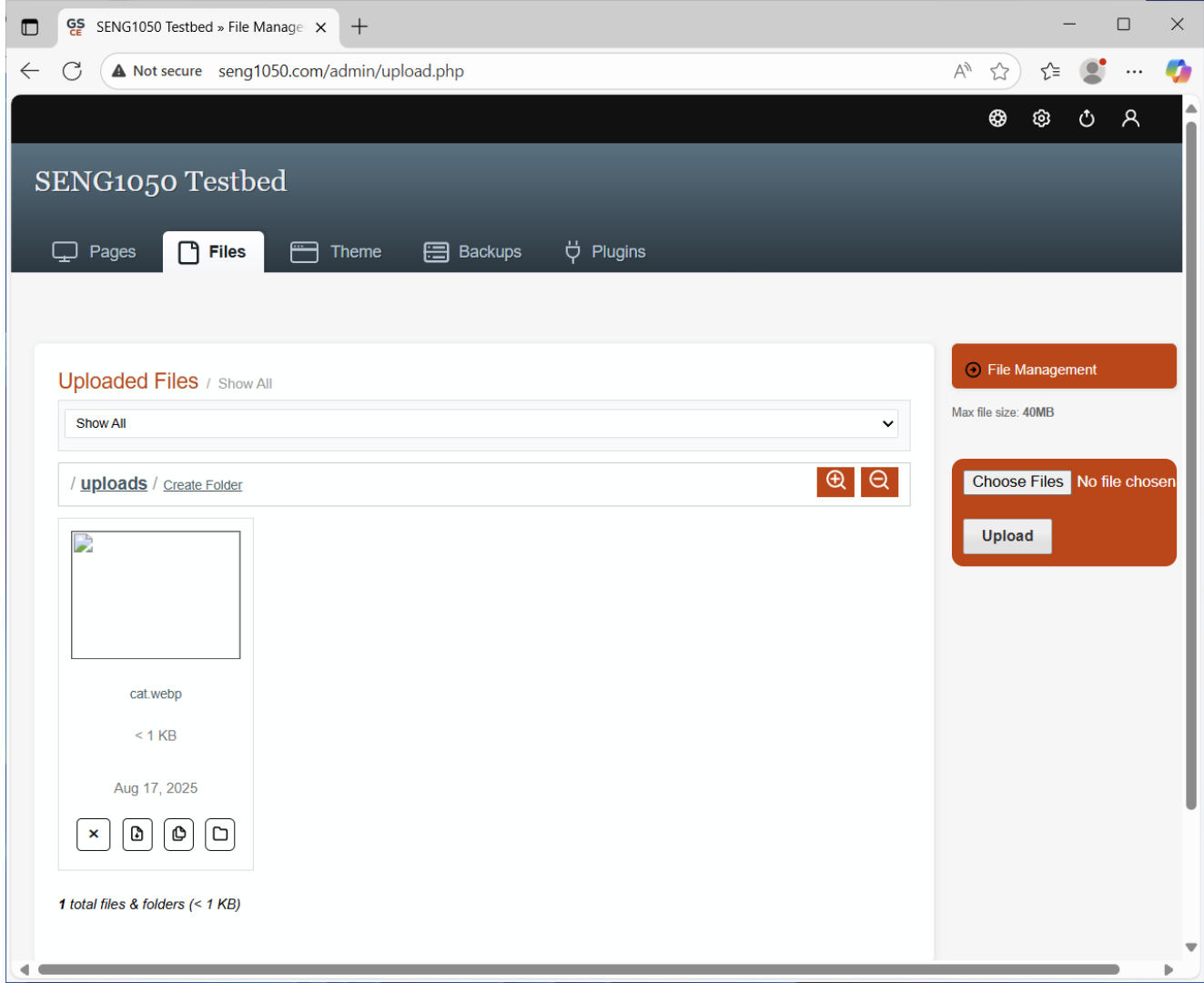
Header 2

Donec venenatis augue. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Integer vulputate pretium augue.

Save Updates

Page last saved by on August 17th, 2025 - 5:02 AM

GetSimple CMS File Management



GetSimple CMS Theme (Template) Management

SENG1050 Testbed > Theme Man. x +

Not secure seng1050.com/admin/theme.php

SENG1050 Testbed

Pages Files Theme Backups Plugins

Choose Your Theme

Theme Folder Location: <http://seng1050.com/theme/W3-Startup/>

W3-Startup Activate Theme

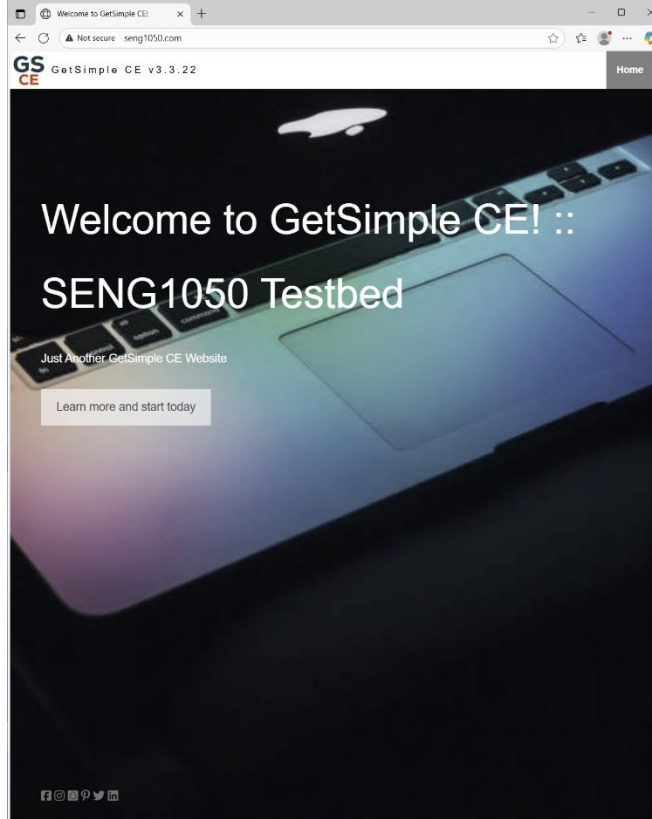
GetSimple CE Starter Theme



W3-Startup

- Choose Theme
- Edit Theme
- Edit Components
- View Sitemap
- Theme Configurator

GetSimple CMS Theme (Template) Switch



Welcome to GetSimple CE!

Thank you for using GetSimple CMS CE. This is your homepage, so please change this text to be what you want.

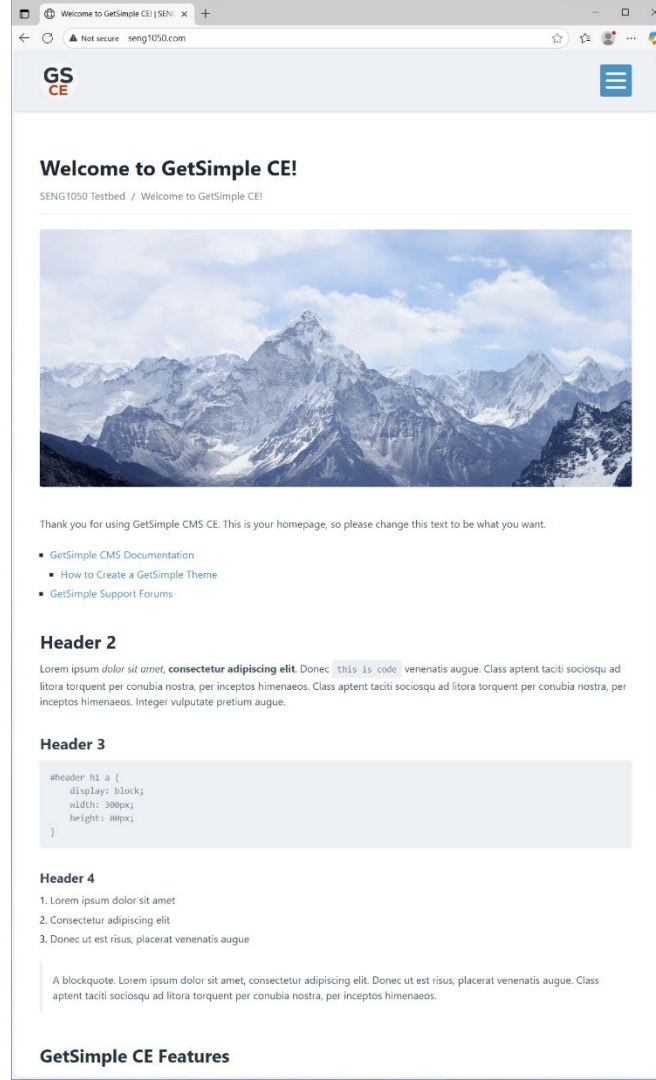
- [GetSimple CMS Documentation](#)
- [How to Create a GetSimple Theme](#)
- [GetSimple Support Forums](#)

Header 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec

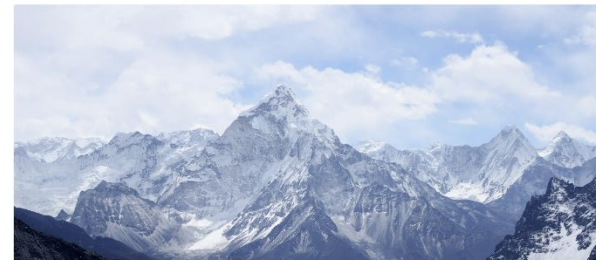


GetSimple CE Features



Welcome to GetSimple CE!

SENG1050 Testbed / Welcome to GetSimple CE!



Thank you for using GetSimple CMS CE. This is your homepage, so please change this text to be what you want.

- [GetSimple CMS Documentation](#)
- [How to Create a GetSimple Theme](#)
- [GetSimple Support Forums](#)

Header 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec `this is code` venenatis augue. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Integer vulputate pretium augue.

Header 3

```
#header h1 a {
  display: block;
  width: 300px;
  height: 80px;
}
```

Header 4

1. Lorem ipsum dolor sit amet
2. Consectetur adipiscing elit
3. Donec ut est risus, placerat venenatis augue

A blockquote. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec ut est risus, placerat venenatis augue. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos.

GetSimple CE Features

Which TWO of the following functions are handled by the Content Management Application of a CMS?

Document management

0%

Templating

0%

Content creation

0%

Compilation

0%

On-access caching

0%



Simple PHP Template



PHP page (index.php)

```
<?php
    $title = 'First PHP Webpage';
    $datafolder = 'data';
    $themefolder = 'theme';
    $content = '
    <h1>First Header</h1>
    <p>Hello, world!</p>
    ';
?>

<?php include($themefolder . '/template.php');?>
```

Template file (template.php stored in theme folder)

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title><?php echo $title; ?></title>
        <link rel="stylesheet" type="text/css"
href="<?php echo $themefolder ?>/style.css" >
    </head>
    <body>
        <?php echo $content; ?>
    </body>
</html>
```

HTML page (index.html)

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>First PHP Page</title>
        <link rel="stylesheet" type="text/css"
href="theme/style.css" >
    </head>
    <body>
        <h1>First Header</h1>
        <p>Hello, world!</p>
    </body>
</html>
```

A Simple PHP Template

Note how the highlighted sections tie to one another.

--- XPath

Example XML File (beatles.xml)

```
<?xml version="1.0"?>
<!-- beatles.xml -->
<beatles>
  <beatle link="http://www.paulmccartney.com">
    <name>
      <firstname>Paul</firstname>
      <lastname>McCartney</lastname>
    </name>
    <plays>bass</plays>
    <hand>left</hand>
  </beatle>
  <beatle link="http://www.johnlennon.com">
    <name>
      <firstname>John</firstname>
      <lastname>Lennon</lastname>
    </name>
    <plays>guitar</plays>
    <hand>right</hand>
  </beatle>
</beatles>
```

XML Output (Boring!!)

- ❑ XML is a standard for the organisation of **well-defined data**
- ❑ XML does not attempt to present data in any visually appealing way
 - ❑ If you open an XML file in your web browser, all you will see is a tree-like structure of the tags.
- ❑ XML is not about visual presentation – rather, it is about sharing of **well-defined data**.
- ❑ **Question:** How can we change a plain XML document into something more visually appealing such as HTML?

```
<!-- beatles.xml -->
<beatles>
  <beatle link="http://www.paulmccartney.com">
    <name>
      <firstname>Paul</firstname>
      <lastname>McCartney</lastname>
    </name>
    <plays>bass</plays>
    <hand>left</hand>
  </beatle>
  <beatle link="http://www.johnlennon.com">
    <name>
      <firstname>John</firstname>
      <lastname>Lennon</lastname>
    </name>
    <plays>guitar</plays>
    <hand>right</hand>
  </beatle>
</beatles>
```

XPath

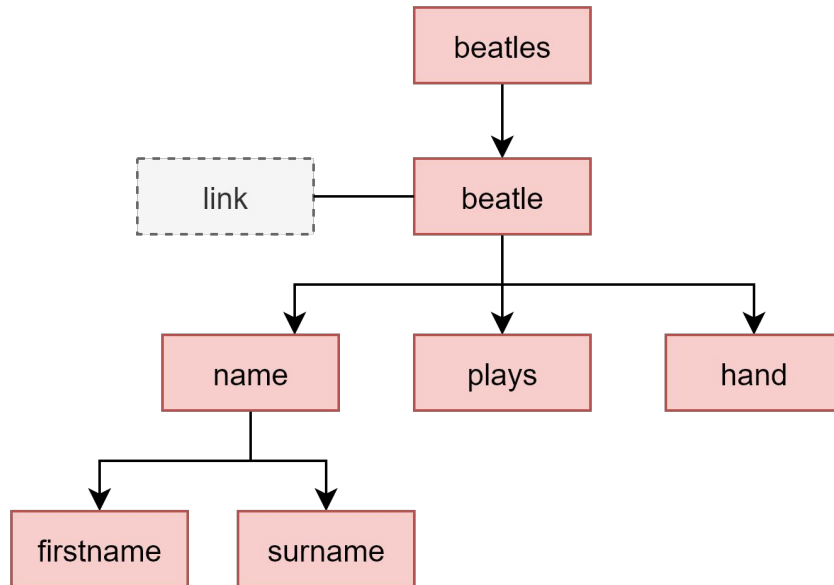
- ❑ XPath provides a way to **navigate** through elements and attributes in XML documents.
- ❑ Both XSLT and PHP support the use of XPath expressions to select elements and attributes in XML documents.
- ❑ XPath contains over 100 built-in functions. Only XPath 1.0 functions are implemented in modern browsers (version 3.0 is current)



Navigating an XML Document

- ❑ `<beatles>` is the root element
- ❑ `<beatle>` is a child element of `<beatles>`
- ❑ `<name>`, `<plays>` and `<hand>` are children elements of `<beatle>`
- ❑ `link` is an attribute node

The relationship between elements can be explained using the terms **parent, child, sibling, ancestor and descendent**.

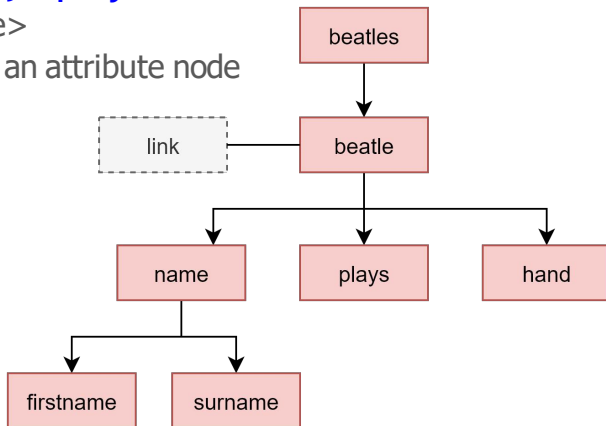




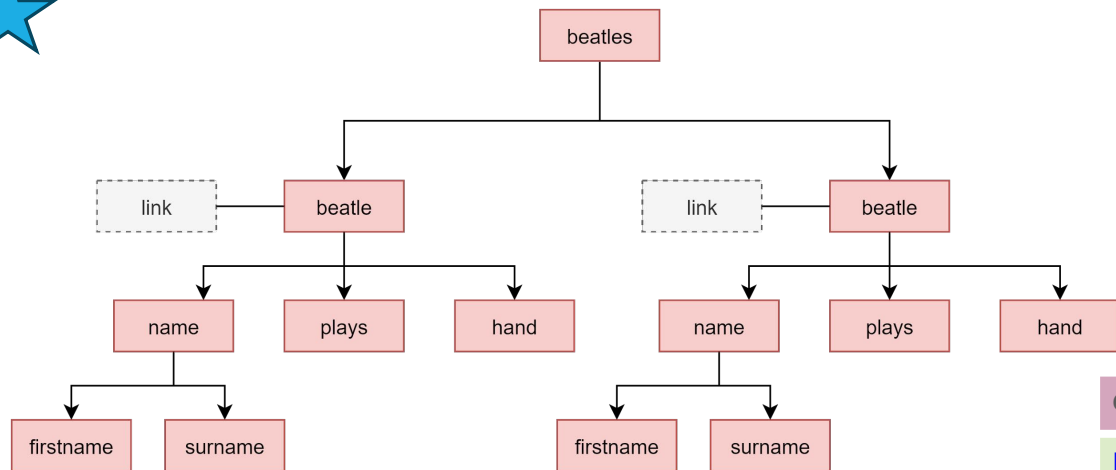
We can select different nodes in an XML tree by using these expressions

XPath Expressions

- ❑ `<beatles>` is the root element
- ❑ `<beatle>` is a child element of `<beatles>`
- ❑ `<name>`, `<plays>` and `<hand>` are child elements of `<beatle>`
- ❑ `link` is an attribute node

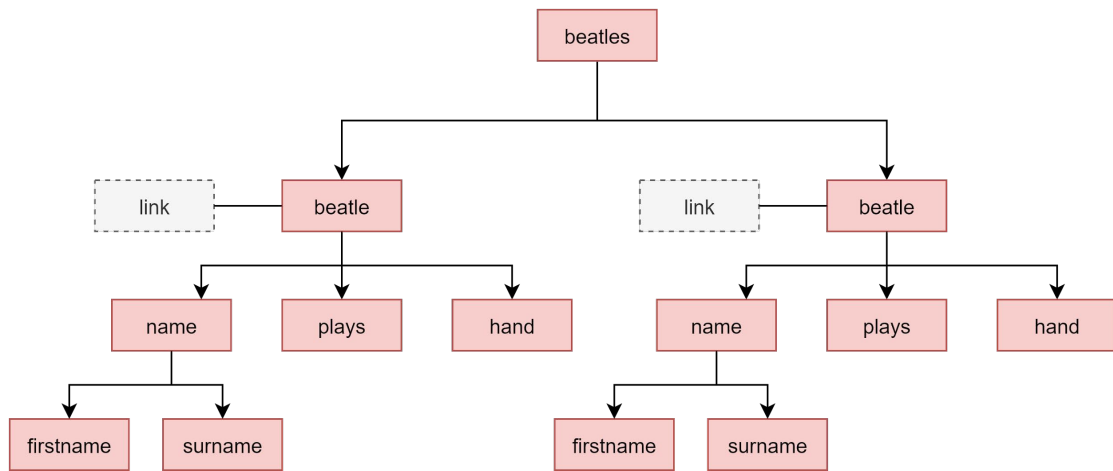


expression	Description
nodename	Select all children with the name "nodename" (relative to context node)
/	Select from the root node (absolute path)
//	Select nodes in the XML from the current node that match the selection no matter where they are
.	Select the current node
..	Select the parent of the current node
@	Select attributes

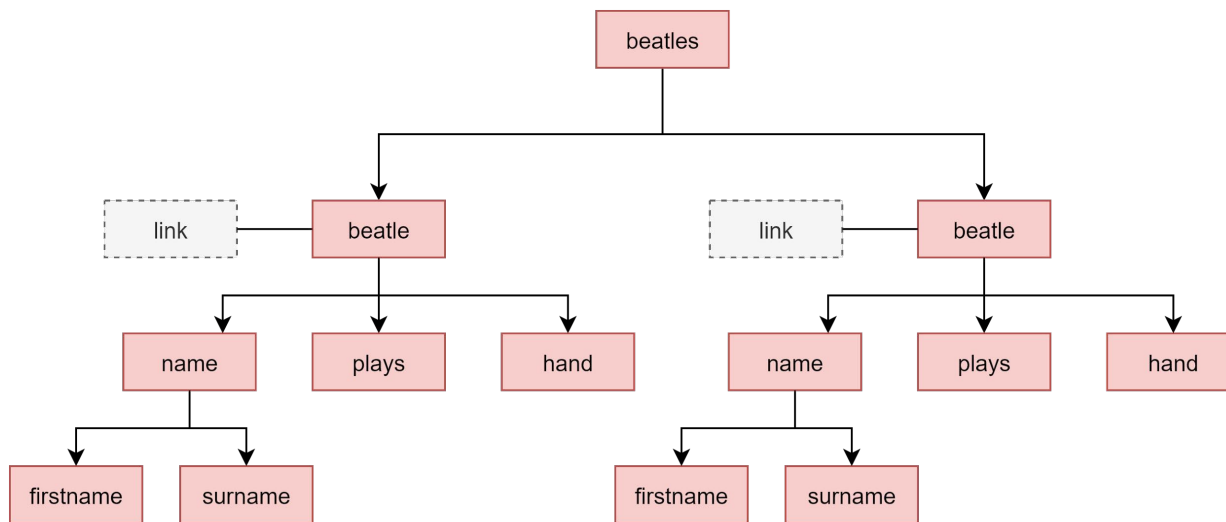


expression	Description
hand	Select all children with the name "hand" (relative to context node)
/beatles	Select from the root element beatles
//beatle	Select all beatle elements no matter where they are
beatles/beatle	Select all beatle elements that are children of beatles (relative to context node)
//@link	Select all attributes that are named link

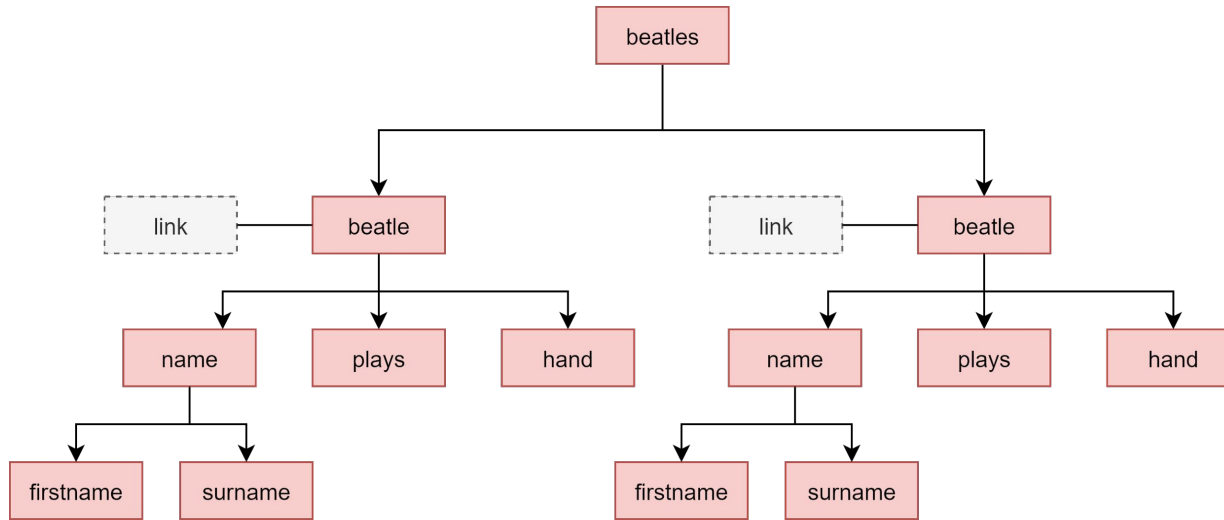
expression	Description
nodename	Select all children with the name "nodename" (relative to context node)
/	Select from the root node
//	Select nodes in the XML from the current node that match the selection no matter where they are
.	Select the current node
..	Select the parent of the current node
@	Select attributes



Path expression	Result
<code>/beatles/beatle[1]</code>	Select the first beatle element
<code>/beatles/beatle[last()]</code>	Select the last beatle element
<code>/beatles/beatle[position() < 3]</code>	Select the first two beatle elements
<code>/beatle[@link]</code>	Select all the beatle elements that have an attribute named link
<code>//beatle[@link='newcastle.edu.au']</code>	Select all beatle elements that have an attribute named link with a value of 'newcastle.edu.au'



Wildcard	Description
*	Matches any element node
@*	Matches any attribute node
/beatles/*	Select all the child nodes of the beatles
//beatle[@*]	Select all beatle elements which have any attribute



Built-in functions	Description
<code>name()</code>	The name of the matched tag
<code>text()</code>	The text of the match
<code>count(nodeset)</code>	The number of elements in the passed nodeset
<code>position()</code>	Return the index position of the node
<code>last()</code>	return the number of items

Consider the provided `beatles.xml` and having the root element as the context (current) node, which of the following XPATH expression will select only the element containing the data "guitar"?

plays

0%

//plays

0%

right

0%

//beatle[position()=2]/plays

0%

/beatles//plays["right"]

0%

```
<?xml version="1.0"?>
<!-- beatles.xml -->

<beatles>
  <beatle link="http://www.paulmccartney.com">
    <name>
      <firstname>Paul</firstname>
      <lastname>McCartney</lastname>
    </name>
    <plays>bass</plays>
    <hand>left</hand>
  </beatle>
  <beatle link="http://www.johnlennon.com">
    <name>
      <firstname>John</firstname>
      <lastname>Lennon</lastname>
    </name>
    <plays>guitar</plays>
    <hand>right</hand>
  </beatle>
</beatles>
```



Writing XML-Connected Templates with PHP / Converting XML to HTML

Arrays

- ❑ Whether it's XPath expressions or accessing the form data in the `'$_POST'` variable, we often need to handle structured data, which is stored in an array.
- ❑ An array is just a variable that holds a collection of data, instead of just one item.
- ❑ In PHP, we can access arrays either by a numerical index, or by a key



Creating Arrays

- ❑ We can use the `array()` function to create a new array
- ❑ Using the `array()` function with nothing inside the parenthesis will create an empty array.
- ❑ An array can store any type of data: numbers, text, objects, even other arrays!

```
$grades = array(55, 75, 22, 99, 4, 68) ;
```

55	75	22	99	4	68
----	----	----	----	---	----



Creating Arrays

- ❑ We can use the `array_push()` function to add an item to the end of an array. We can add multiple items at the same time.
- ❑ Similarly, the `array_pop()` function will remove and return the last item from an array.

```
$grades = array() ;  
array_push($grades, 55);  
array_push($grades, 75, 22, 99, 4);  
array_push($grades, 68);
```

55	75	22	99	4	68
----	----	----	----	---	----

```
$last_element = array_pop($grades); // 68
```

55	75	22	99	4
----	----	----	----	---



Accessing Array Elements

- ❑ The most common way of reading and writing to an array is by use of a numerical index. The first element of the array is located at index '0', and we count up from there. **WARNING: XPATH expressions start counting from '1' instead!**
- ❑ Indexes can be used to both read a value and write a value to an element in the array

```
$grades = array() ;  
array_push($grades, 55);  
array_push($grades, 75, 22, 99, 4);  
array_push($grades, 68);
```

55	75	22	99	4	68
----	----	----	----	---	----

```
$student = $grades[2]; // 22  
$newGrade = $student + 10;  
$grades[2] = $newGrade;
```

55	75	32	99	4	68
----	----	----	----	---	----

Indexes - 0 1 2 3 4 5



Accessing Array Elements

- ❑ In PHP, array elements can also be given a name, which can be used to access them (called a 'key'. This is called an 'associative array'.
- ❑ The `$_POST` variable is an example of an associative array. This is why we give each 'input' element of a form a 'name' attribute with a value, this becomes the key for that element in the array.

```
$grades = array() ;  
$grades["Alice"] = 55;  
$grades["Bob"] = 22;
```

55	22
----	----

Keys - Alice Bob

```
$alice = $grades["Alice"]; // 55  
$newGrade = $alice + 10;  
$grades["Alice"] = $newGrade;
```

65	22
----	----

Keys - Alice Bob



Looping over an Array

- We will often want to read/write to each element of an array in turn. To do this, we can use a special type of loop in PHP known as the 'for each' loop:

```
foreach (array as arrayelement) {  
    // loop body  
}
```

```
$grades = array() ;  
array_push($grades, 55);  
array_push($grades, 75, 22, 99, 4);  
array_push($grades, 68);
```

55	75	22	99	4	68
----	----	----	----	---	----

```
foreach ($grades as $mygrade) {  
    echo '<p>' . $mygrade . '</p>';  
}
```

55
75
22
99
4
68



Loading an XML file in PHP

- ❑ PHP: Use the function:
`simplexml_load_file(filename);`

Neither the XML file nor template are sent to the client, only the resulting HTML page.

```
<?php
$xml = simplexml_load_file("beatles.xml");
?>
```

```
<?xml version="1.0"?>
<!-- beatles.xml -->

<beatles>
  <beatle link="http://www.paulmccartney.com">
    <name>
      <firstname>Paul</firstname>
      <lastname>McCartney</lastname>
    </name>
    <plays>bass</plays>
    <hand>left</hand>
  </beatle>
  <beatle link="http://www.johnlennon.com">
    <name>
      <firstname>John</firstname>
      <lastname>Lennon</lastname>
    </name>
    <plays>guitar</plays>
    <hand>right</hand>
  </beatle>
</beatles>
```



Selecting XML Elements in PHP

- ❑ PHP: Each node's value is an attribute which can be accessed using the -> operator. Alternatively, for even greater control, you can use the xpath() function:
`$xml->xpath("XPath expression");`

```
<?php
$xml = simplexml_load_file("beatles.xml");
$xml->xpath("//beatle");
?>
```

```
<?xml version="1.0"?>
<!-- beatles.xml -->

<beatles>
  <beatle link="http://www.paulmccartney.com">
    <name>
      <firstname>Paul</firstname>
      <lastname>McCartney</lastname>
    </name>
    <plays>bass</plays>
    <hand>left</hand>
  </beatle>
  <beatle link="http://www.johnlennon.com">
    <name>
      <firstname>John</firstname>
      <lastname>Lennon</lastname>
    </name>
    <plays>guitar</plays>
    <hand>right</hand>
  </beatle>
</beatles>
```



Selecting XML Attributes in PHP

- ❑ PHP: Same as with elements.

```
<?php
$xml = simplexml_load_file("beatles.xml");
foreach ($xml->xpath("//@link") as $link) {
    echo '<p>' . $link . '</p>';
}
?>
```

- ❑ However, if accessing directly, an element's attributes may be accessed as part of an array, e.g.,:

```
<?php
$xml = simplexml_load_file("beatles.xml");
foreach ($xml->children() as $beatle) {
    echo '<p>' . $beatle['link'] . '</p>';
}
?>
```

```
<?xml version="1.0"?>
<!-- beatles.xml -->

<beatles>
  <beatle link="http://www.paulmccartney.com">
    <name>
      <firstname>Paul</firstname>
      <lastname>McCartney</lastname>
    </name>
    <plays>bass</plays>
    <hand>left</hand>
  </beatle>
  <beatle link="http://www.johnlennon.com">
    <name>
      <firstname>John</firstname>
      <lastname>Lennon</lastname>
    </name>
    <plays>guitar</plays>
    <hand>right</hand>
  </beatle>
</beatles>
```



Conditions in Templates: PHP if-else

```
<?php
$xml = simplexml_load_file("some.xml");
if ((int) $xml->xpath("//BBB[2]")[0] > (int)
$xml->xpath("//BBB[3]")[0]) {
    echo "<p>10 &gt; 5</p>";
}
if ($xml->xpath("//BBB[1]")) {
    echo "<p>It exists</p>";
}
?>
```

(int) forces the result to be treated as an integer (instead of text)

```
<AAA>
  <BBB>abc</BBB>
  <BBB>10</BBB>
  <BBB>5</BBB>
</AAA>
```



10 > 5
It exists



Conditions in Templates: PHP if-else

```
<?php
$xml = simplexml_load_file("some.xml");
if ((int) $xml->xpath("//BBB[2]")[0] > (int)
$xml->xpath("//BBB[3]")[0]) {
    echo "<p>10 &gt; 5</p>";
}
if ($xml->xpath("//BBB[1]")) {
    echo "<p>It exists</p>";
}
?>
```

The `xpath` function
always returns an array,
so we need to select the
first element of the array

```
<AAA>
  <BBB>abc</BBB>
  <BBB>10</BBB>
  <BBB>5</BBB>
</AAA>
```



10 > 5
It exists

Conditions in Templates: PHP switch

```
<?php
$xml = simplexml_load_file('sample.xml');
foreach ($xml->children() as $child) {
    echo '<p>' . $child . ':';
    switch (true) {
        case $child < 10:
            echo 'Less than 10';
            break;
        case $child > 10:
            echo 'More than 10';
            break;
        default:
            echo 'Not a number';
            break;
    }
    echo '</p>';
}
?>
```

```
<AAA>
  <BBB>abc</BBB>
  <BBB>11</BBB>
  <BBB>7</BBB>
</AAA>
```

abc: Not a number
11: More than 10
7: Less than 10

In a switch block, the switch expression is compared with each case expression using the equality operator, the first matching case will run until reaching a break statement or the end of the entire switch block.



Looping in Templates: PHP

❑ PHP: Use a while, do-while, for, or foreach loop.

```
<?php
$xml = simplexml_load_file("beatles.xml");
foreach($xml->children() as $child) {
    echo "<p>" . $child . "</p>";
}
?>
```

```
<AAA>
  <BBB>cc</BBB>
  <BBB>dd</BBB>
  <BBB>ff</BBB>
  <BBB>aa</BBB>
  <BBB>ee</BBB>
</AAA>
```

cc
dd
ff
aa
ee

Sorting Elements

- ❑ PHP: One of the few cases where PHP is slightly more complicated as the `children()` function doesn't return a sortable array – We have two choices:
 1. Cast to an array, then sort.
 2. Just use the `xpath()` function instead, as this returns a sortable array.

Sorting Elements: PHP (Long Way)

```
<?php
$xml = simplexml_load_file('zoo.xml');
foreach ($xml->children() as $child) {
    echo '<h1>' . $child->getName() . '</h1>';
    echo '<ul>';
    $sortable_child = array();
    foreach ($child->children() as $sortchild) {
        $sortable_child[] = $sortchild;
    }
    usort($sortable_child, function($x, $y) {
        return ((int) $y['pop'] - (int) $x['pop']);
    });
    foreach ($sortable_child as $sorted) {
        echo '<li>' . $sorted->getName() . '(' . $sorted['pop'] . ')</li>';
    }
    echo '</ul>';
}
?>
```

Cast to an array

Perform the sort

```
<zoo>
  <birds>
    <albatross pop="4" />
    <buzzard pop="2" />
    <chickadee pop="12" />
  </birds>
  <mammals>
    <aardvark pop="5" />
    <bat pop="200" />
    <cheetah pop="2" />
  </mammals>
</zoo>
```

birds

- chickadee (12)
- albatross (4)
- buzzard (2)

mammals

- bat (200)
- aardvark (5)
- cheetah (2)

Sorting Elements: PHP (Short Way)

```
<?php
$xml = simplexml_load_file('zoo.xml');
foreach ($xml->children() as $child) {
    echo '<h1>' . $child->getName() . '</h1>';
    echo '<ul>';
    $sortable_child = $child->xpath("./*");
    usort($sortable_child, function($x, $y) {
        return ((int) $y['pop'] - (int) $x['pop']);
    });
    foreach ($sortable_child as $sorted) {
        echo '<li>' . $sorted->getName() . '(' . $sorted['pop'] . ')</li>';
    }
    echo '</ul>';
}
?>
```

Cast to an array

Perform the sort

The `usort()` function allows you to set your own sorting rule (which needs to define an answer to the question of which element in a pair is larger). In this case, we are comparing the 'pop' attributes of each element and placing them in descending order.

```
<zoo>
  <birds>
    <albatross pop="4" />
    <buzzard pop="2" />
    <chickadee pop="12" />
  </birds>
  <mammals>
    <aardvark pop="5" />
    <bat pop="200" />
    <cheetah pop="2" />
  </mammals>
</zoo>
```

birds

- chickadee (12)
- albatross (4)
- buzzard (2)

mammals

- bat (200)
- aardvark (5)
- cheetah (2)

Adding data to XML files

- ❑ Several methods of SimpleXML objects can be used to add data to a loaded XML file:
 - ❑ `addChild(name, value)` – Add an element to the current node with the given name and value.
 - ❑ `addAttribute(name, value)` – Add an attribute to the current node with the given name and value.
 - ❑ `saveXML(filename)` – Call this on the root node to save the entire updated XML document (including the DTD and any comments) to the given filename.

```
<root>  
  <mytag></mytag>  
</root>
```



```
<?php  
$filename = 'sample.xml';  
$xml = simplexml_load_file($filename);  
foreach ($xml->children() as $child) {  
    $child->addAttribute("Hello", "World");  
    $child->addChild("NewChild", "Hi!");  
}  
$xml->saveXML($filename);  
?>
```



```
<!-- After the script runs: -->  
<root>  
  <mytag Hello="World">  
    <NewChild>Hi!</NewChild>  
  </mytag>  
</root>
```

PHP Templates: File Operations

- ❑ Several additional functions in PHP allow us to manage files:
 - ❑ `file_exists(filename)` – Check if a file exists, returns true or false
 - ❑ `copy(source, destination)` – Copy a file from source to destination, if the destination file already exists, it will be overwritten. Returns true on success and false on failure.
 - ❑ `rename(source, destination)` – Moves a file from source to destination, renaming it if a different filename is used. If the destination file already exists, it will be overwritten. Returns true on success and false on failure.



XML to HTML: A Simple Template

PHP page (xmltest.php)

```
<?php
    $title = 'First XML Webpage';
    $datafolder = 'data';
    $themefolder = 'theme';
    $content = '';
?>

<?php include($themefolder .
'/xml-template.php'); ?>
```

Template file (xml-template.php stored in theme folder)

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title><?php echo $title; ?></title>
        <link rel="stylesheet" type="text/css" href="<?php echo
$themefolder ?>/style.css" >
    </head>

    <body>
        <?php
            $xmlpath = $datafolder . "/xml/" . $_GET["name"] . ".xml";
            $root = simplexml_load_file($xmlpath);
            foreach ($root->xpath("//@link") as $link) {
                echo '<p>' . $link . '</p>';
            }
        ?>
    </body>
</html>
```



XML to HTML: A Simple Template

Access from: <http://www.demo.com/xmltest.php?name=beattles>

Template file (xml-template.php stored in theme folder)

Data file (beattles.xml stored in /data/xml)

```
<beatles>
  <beatle
    link="http://www.paulmccartney.com">
      <name>
        <firstname>Paul</firstname>
        <lastname>McCartney</lastname>
      </name>
      <plays>bass</plays>
      <hand>left</hand>
    </beatle>
    <beatle link="http://www.johnlennon.com">
      <name>
        <firstname>John</firstname>
        <lastname>Lennon</lastname>
      </name>
      <plays>guitar</plays>
      <hand>right</hand>
    </beatle>
  </beatles>
```

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title><?php echo $title; ?></title>
    <link rel="stylesheet" type="text/css" href="<?php echo
$themefolder ?>/style.css" >
  </head>

  <body>
    <?php
      $xmlpath = $datafolder . "/xml/" . $_GET["name"] . ".xml";
      $root = simplexml_load_file($xmlpath);
      foreach ($root->xpath("//@link") as $link) {
        echo '<p>' . $link . '</p>';
      }
    ?>
  </body>
</html>
```



Summary - Creating our website

- ❑ We were addressing the problem of scalability and looking at how we can pull data from our XML files to insert into HTML documents.
- ❑ As websites grow, managing the increased content becomes too much for a human, and non-technical users may also need to add content to the website. A content management system abstracts the technical elements of the website and organises content.
- ❑ Content management systems include web templating systems that allow common HTML to be written once in a template, and have data inserted via placeholder functions.
- ❑ We looked at how PHP and XSLT can be used for creating templates and compared server-side templating to client-side templating. We can use this to get data from our XML files and insert it into HTML documents.
- ❑ The next step is to complete the separation of concerns on our website. So far, we have separated the content, structure, styling, and server-side behaviour; the final step is to separate the client-side behaviour using JavaScript – next week.

References

❏ **Programming the World Wide Web (8e)**

By Robert Sebesta

📖 *Chapter 7*

📖 *Chapter 9*